

DEVOPS PRACTICAL REFERENCE SERIES

PART 03

Networking, DNS, HTTP/HTTPS & SSL/TLS

From OSI layers and subnetting to TLS handshakes and load-balancer health checks -- the complete networking foundation a DevOps engineer needs before Docker and Kubernetes.

[Networking](#)[DNS](#)[HTTP/HTTPS](#)[TLS/SSL](#)[Load Balancing](#)[Troubleshooting](#)

Audience: DevOps engineers with ~2 years of experience

Format: Deep-dive practical reference — concepts, architecture, configuration, commands, troubleshooting, security, automation, and hands-on labs.

How to use: Read sequentially. Each part is self-contained but builds on earlier parts in the series.

Table of Contents

1. Introduction	5
1.1 What This Part Covers	5
1.2 Why This Matters for DevOps	5
2. The OSI Model and TCP/IP in Practice	6
2.1 What It Is and Why It Exists	6
2.2 The Layers, Mapped to Real DevOps Concepts	6
2.3 Encapsulation: What Actually Happens to Your Data	7
3. IP Addressing and Subnetting	8
3.1 IPv4 Addressing	8
3.2 CIDR Notation and Subnetting	8
3.3 Subnetting Math, Worked Example	9
3.4 Private (RFC 1918) Address Ranges and NAT	9
3.5 IPv6, Briefly	10
4. TCP vs UDP	11
4.1 What They Are and Why Both Exist	11
4.2 The TCP Three-Way Handshake	11
4.3 TCP Connection States	12
4.4 Why DNS Mostly Uses UDP (and Sometimes TCP)	12
5. Routing and Switching Basics	13
5.1 What It Is and Why It Matters	13
5.2 Default Gateways and Route Tables	13
5.3 ARP: Resolving IP to MAC Addresses	13
5.4 Security Groups vs Network ACLs (the Conceptual Preview)	14
6. DNS Deep Dive	15
6.1 What It Is and Why It Exists	15
6.2 DNS Hierarchy and Resolution Process	15
6.3 DNS Record Types	16
6.4 TTL and Caching	17
6.5 Authoritative DNS in Practice: Route 53 and Friends	17
6.6 Split-Horizon / Private DNS	18
7. HTTP and HTTPS Protocol Deep Dive	19
7.1 What It Is and Why It Exists	19
7.2 Anatomy of an HTTP Request and Response	19
7.3 HTTP Methods	20
7.4 HTTP Status Codes	21
7.5 Essential Headers	22
7.6 HTTP/1.1 vs HTTP/2 vs HTTP/3	23

7.7 Keep-Alive and Connection Reuse	23
7.8 HTTP Caching	24
8. SSL/TLS and Certificates	25
8.1 What It Is and Why It Exists	25
8.2 The TLS Handshake	25
8.3 The Certificate Chain of Trust	26
8.4 Certificate Authorities and ACME / Let's Encrypt	27
8.5 Self-Signed Certificates	27
8.6 Common Certificate Problems	28
8.7 mTLS (Mutual TLS)	28
8.8 TLS Termination Points	29
9. Load Balancing and Reverse Proxies	30
9.1 What They Are and Why They Exist	30
9.2 Why This Architecture Exists	30
9.3 Load Balancing Algorithms	31
9.4 Health Checks	31
9.5 Session Affinity ("Sticky Sessions")	32
9.6 API Gateways	32
10. Network Troubleshooting Methodology	33
10.1 The Layered Diagnostic Approach	33
10.2 The Systematic Sequence	33
10.3 Common Production Scenarios	34
Scenario: Intermittent connection failures under load	34
Scenario: Works from one machine, fails from another	34
Scenario: Load balancer shows healthy targets, but users still report errors	34
11. Network Security Fundamentals	36
11.1 Defense in Depth	36
11.2 Common Network-Level Attacks and Mitigations	37
11.3 TLS Everywhere and HSTS	37
11.4 VPNs and Private Connectivity, Conceptually	37
11.5 Zero Trust, Conceptually	38
12. Hands-On Labs	39
Lab 1 — Subnetting by Hand, Then Verify	39
Lab 2 — Watch DNS Resolution Happen	39
Lab 3 — TLS Handshake and Certificate Chain Inspection	39
Lab 4 — HTTP Status Code and Header Investigation	40
Lab 5 — Full-Stack Network Troubleshooting Simulation	40
13. Command & Reference Cheat Sheet	41
13.1 DNS	41
13.2 Connectivity	41

13.3 HTTP/TLS	41
13.4 Subnetting Quick Reference	42
14. Production Readiness Checklist	43

1. Introduction

1.1 What This Part Covers

Part 03 covers the networking fundamentals that sit underneath everything else in this series: how packets actually move between hosts, how DNS turns names into addresses, how HTTP/HTTPS actually work on the wire, and how TLS establishes trust and encryption. Part 01 gave you the Linux-side *tools* (`ip` , `ss` , `curl` , `dig` , `tcpdump`); this part gives you the *protocol knowledge* those tools are inspecting — without which “the connection timed out” is just a scary phrase instead of something you can systematically localize to a specific layer and fix.

HOW TO USE THIS PART

This part is intentionally protocol-focused rather than tool-focused — Part 01, Section 8 already covers the Linux command-line tools (`dig` , `curl` , `tcpdump` , `ss`) in depth; this part explains WHAT those tools are actually showing you and WHY the protocols behave the way they do. If you already understand TCP/IP and HTTP deeply, skim Sections 2-5 and focus on Section 8 (TLS) and Section 11 (troubleshooting), which is where most 2-3 year engineers still have real gaps.

1.2 Why This Matters for DevOps

Every outage you’ll ever debug involves a request going from somewhere to somewhere else and failing at some specific layer: DNS didn’t resolve, the TCP handshake never completed, the TLS certificate was expired, the load balancer’s health check was hitting the wrong port, the HTTP response had the wrong `Content-Length` . Kubernetes Services, Ingress controllers, load balancers (AWS ALB/NLB, covered in Part 20), and CDNs are all just structured applications of these fundamental protocols — understanding them directly is what lets you reason about a new, unfamiliar piece of infrastructure instead of needing a tutorial for every new tool you touch.

2. The OSI Model and TCP/IP in Practice

2.1 What It Is and Why It Exists

The OSI model is a conceptual framework describing network communication as seven stacked layers, each with a distinct responsibility, communicating only with the layers directly above and below it. In practice, real-world networking follows the simpler, four-layer **TCP/IP model**, but the OSI model's vocabulary ("Layer 3," "Layer 7," "L4 load balancer") is universal in production networking and infrastructure discussions — you need to speak this language fluently.

2.2 The Layers, Mapped to Real DevOps Concepts

OSI Layer with	TCP/IP Layer	What DevOps actually calls this / deals
7. Application balancing," Ingress rules	Application	HTTP, DNS, SSH, gRPC – "L7 load
6. Presentation discussed separately in practice)		TLS/encryption, data encoding (rarely
5. Session into L7 discussions today)		Session establishment (often folded
4. Transport ports, the 3-way handshake	Transport	TCP, UDP – "L4 load balancing,"
3. Network route tables, CIDR blocks	Internet	IP addressing, routing – "L3," VPC
2. Data Link VLANs	Network Access	MAC addresses, switches, ARP,
1. Physical cloud engineer's concern directly		Cables, fiber, radio – rarely a

Layer	Unit of data	Key devices/concepts	Example DevOps artifact
L7 Application	Message/request	HTTP, DNS, SMTP	Kubernetes Ingress, NGINX reverse proxy rules
L4 Transport	Segment (TCP) / Datagram (UDP)	Ports, TCP handshake, flow control	AWS Network Load Balancer, Kubernetes <code>Service</code> (ClusterIP)
L3 Network	Packet	IP addressing, routing	VPC CIDR blocks, route tables, <code>NetworkPolicy</code>
L2 Data Link	Frame	MAC addresses, switches	VPC ENIs, CNI bridge networking

WHY “L4 VS L7 LOAD BALANCER” IS A REAL, IMPORTANT DISTINCTION

An L4 load balancer (e.g., AWS NLB) makes routing decisions using ONLY IP address and port — it doesn’t parse HTTP at all, so it can balance ANY TCP traffic (databases, raw TCP services) with extremely low latency, but it can’t route based on URL path or hostname. An L7 load balancer (e.g., AWS ALB, NGINX, Kubernetes Ingress) actually terminates and parses the HTTP request, so it CAN route `/api/*` to one backend and `/static/*` to another, inspect headers, and terminate TLS — but it only understands HTTP-family protocols. This single distinction explains why both types exist and when each is the right infrastructure choice (Part 20 covers the AWS-specific implementations in full depth).

2.3 Encapsulation: What Actually Happens to Your Data

As data moves down the stack to be sent, each layer wraps (encapsulates) the layer above’s data with its own header:

```

Application data:      [ HTTP request: "GET /api/users HTTP/1.1..." ]
                        | wrapped by Transport layer
TCP segment:          [TCP header: src port, dst port, seq#...][ HTTP data ]
                        | wrapped by Network layer
IP packet:             [IP header: src IP, dst IP, TTL...][TCP header][ HTTP data ]
                        | wrapped by Data Link layer
Ethernet frame: [MAC header: src MAC, dst MAC][IP header][TCP header][ HTTP data ]

```

At the receiving end, this process reverses — each layer strips its own header and passes the payload up. This is precisely why `tcpdump` (Part 01, Section 8.6) can show you MAC addresses, IPs, ports, AND the actual HTTP content all in one capture: it’s all literally nested inside the same frame on the wire.

3. IP Addressing and Subnetting

3.1 IPv4 Addressing

An IPv4 address is a 32-bit number, written as four 8-bit octets (0-255) separated by dots: `192.168.1.10`. Every network interface that communicates on an IP network needs one.

3.2 CIDR Notation and Subnetting

CIDR (Classless Inter-Domain Routing) notation expresses a network's size as `<address>/<prefix-length>` — the prefix length is how many leading bits are the fixed “network” portion, with the rest available for individual hosts.

```
10.0.0.0/16
└─ first 16 bits are the NETWORK, remaining 16 bits are available for HOSTS
└─ 2^16 = 65,536 total addresses in this block (65,534 usable – see below)

10.0.1.0/24
└─ first 24 bits are the NETWORK, remaining 8 bits for HOSTS
└─ 2^8 = 256 total addresses (254 usable)
```

CIDR prefix	Total addresses	Usable hosts (minus network + broadcast)	Common use
/32	1	1 (a single host route)	A specific host, a VPN client route
/30	4	2	Point-to-point links
/28	16	14	A small subnet (a handful of servers)
/24	256	254	A typical “small” subnet — one per AZ in many VPC designs
/16	65,536	65,534	A typical whole-VPC CIDR block
/8	16,777,216	16,777,214	Entire large private range (e.g., <code>10.0.0.0/8</code>)

WHY “USABLE HOSTS” IS TOTAL MINUS 2

In every IPv4 subnet, the **FIRST** address (all host bits 0) is the **network address** (identifies the subnet itself, not assignable to a host) and the **LAST** address (all host bits 1) is the **broadcast address** (reaches every host on that subnet) — neither can be assigned to an individual host, which is why a /24 has 256 total addresses but only 254 usable ones.

Cloud VPC subnets (AWS, in particular) additionally reserve 3 more addresses for internal networking use, so an AWS /24 subnet actually has only 251 usable addresses — a detail that catches people sizing subnets for exactly 254 hosts.

3.3 Subnetting Math, Worked Example

“I have `10.0.0.0/16` and need to split it into 4 equal subnets for 4 availability zones.”

```
Starting block: 10.0.0.0/16 (65,536 addresses)
Splitting into 4 equal parts means borrowing 2 bits ( $2^2 = 4$ ) from the host portion:
/16 + 2 bits = /18 for each subnet (16,384 addresses each)
```

```
10.0.0.0/18 → 10.0.0.0 to 10.0.63.255 (AZ-a)
10.0.64.0/18 → 10.0.64.0 to 10.0.127.255 (AZ-b)
10.0.128.0/18 → 10.0.128.0 to 10.0.191.255 (AZ-c)
10.0.192.0/18 → 10.0.192.0 to 10.0.255.255 (AZ-d)
```

```
# Let a tool do the subnet math in practice – don't do this by hand under incident
pressure
ipcalc 10.0.0.0/18 # shows network, broadcast, host range, netmask
(install: apt install ipcalc)
sipcalc 10.0.0.0/16 -s 4 # split a block into N equal subnets
automatically
```

3.4 Private (RFC 1918) Address Ranges and NAT

Range	CIDR	Typical use
<code>10.0.0.0</code> - <code>10.255.255.255</code>	<code>10.0.0.0/8</code>	Large private networks — the default for most cloud VPCs
<code>172.16.0.0</code> - <code>172.31.255.255</code>	<code>172.16.0.0/12</code>	Private networks — Docker's default bridge network range lives here
<code>192.168.0.0</code> - <code>192.168.255.255</code>	<code>192.168.0.0/16</code>	Small/home private networks

These ranges are **not routable on the public internet** — any traffic from a private IP destined for the internet must go through **NAT (Network Address Translation)**, which rewrites the source IP to a public one at the network's edge (an AWS NAT Gateway, for example). This is exactly why a private EC2 instance can reach the internet outbound (via a NAT Gateway) but cannot be reached FROM the internet without an explicit public-facing load balancer or a public IP assignment.

```
# See your own effective public IP, as the internet sees it (useful to confirm NAT/
egress IP)
curl -s ifconfig.me
curl -s https://checkip.amazonaws.com
```

3.5 IPv6, Briefly

IPv6 uses 128-bit addresses (vs IPv4's 32-bit), written as eight groups of hex digits (`2001:0db8:85a3:0000:0000:8a2e:0370:7334` , commonly abbreviated `2001:db8:85a3::8a2e:370:7334` using `::` to collapse consecutive zero groups once). It exists primarily to solve IPv4 address exhaustion — there is no NAT requirement by design (every device can have a globally routable address), though NAT66/firewalling is still commonly used for security reasons in practice. Most cloud providers support dual-stack (IPv4+IPv6) VPCs today; IPv6-only backends are still relatively uncommon in mainstream enterprise DevOps work but increasingly offered as an option worth knowing exists.

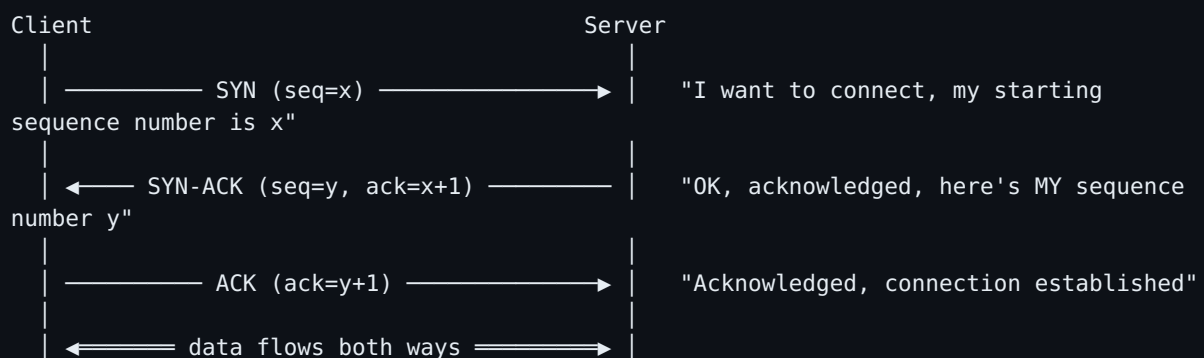
4. TCP vs UDP

4.1 What They Are and Why Both Exist

TCP and UDP are both **Layer 4 (Transport)** protocols sitting on top of IP, but they make fundamentally different tradeoffs between reliability and speed — nearly everything you build sits on top of one or the other.

	TCP	UDP
Connection	Connection-oriented (handshake required before data flows)	Connectionless (just send)
Reliability	Guaranteed delivery, in-order, automatic retransmission	No guarantees — packets can be lost, duplicated, or arrive out of order
Overhead	Higher (headers, acknowledgments, flow control)	Minimal
Use cases	HTTP/HTTPS, SSH, databases, anything needing reliable delivery	DNS (for most queries), video/voice streaming, gaming, metrics (e.g., StatsD)

4.2 The TCP Three-Way Handshake



```
# Watch the handshake happen in a live capture
sudo tcpdump -i any -n 'tcp[tcpflags] & (tcp-syn|tcp-ack) != 0' port 443
```

WHY “CONNECTION REFUSED” VS “TIMEOUT” MEAN DIFFERENT THINGS

A **“connection refused”** error means the SYN packet reached the destination host and got a definitive answer: nothing is listening on that port (the kernel sent back a **RST**, reset, packet) — the host is reachable, networking is fine, but no service is bound to that port. A **“connection timed out”** means the SYN packet was sent but NO response came back at all within the timeout window — this points to a firewall/security-group silently dropping

the packet, a routing problem, or the host being genuinely unreachable. This distinction alone tells you whether to go look at the application (refused) or the network path/firewall rules (timeout) FIRST — one of the most useful diagnostic shortcuts in all of networking troubleshooting.

4.3 TCP Connection States

```
ss -tan # see live TCP connections and their states on this host
(Part 01, Section 8.3)
```

State	Meaning
LISTEN	A server socket waiting for incoming connections
SYN_SENT	Client has sent SYN, waiting for SYN-ACK
SYN_RECV	Server received SYN, sent SYN-ACK, waiting for final ACK
ESTABLISHED	Handshake complete, data can flow
FIN_WAIT_1 / FIN_WAIT_2	This side has initiated connection close
CLOSE_WAIT	The OTHER side closed; this side hasn't closed yet — a large, growing number of these indicates an application bug that isn't closing its sockets
TIME_WAIT	Connection closed, waiting (typically ~60s) to ensure any stray late packets are handled before the port can be reused

COMMON PRODUCTION ISSUE: PORT/SOCKET EXHAUSTION

A host or load balancer generating huge numbers of short-lived outbound connections can exhaust ephemeral ports or accumulate excessive `TIME_WAIT` sockets, eventually causing new connections to fail. `ss -tan state time-wait | wc -l` and `ss -tan state close-wait | wc -l` are the first diagnostic commands when “random connection failures under load” is reported — a high `CLOSE_WAIT` count specifically points at the application failing to close connections/sockets properly, not a network issue at all.

4.4 Why DNS Mostly Uses UDP (and Sometimes TCP)

A standard DNS query is a single small request/response — the overhead of a full TCP handshake isn't worth it for most lookups, so DNS uses UDP by default (port 53). DNS falls back to TCP (also port 53) when the response is too large for a single UDP packet (historically >512 bytes, extended by EDNS0 today) — common with DNSSEC-signed responses or zone transfers between DNS servers. This is exactly why both TCP and UDP port 53 need to be open for DNS to work reliably in a firewall/security-group configuration.

5. Routing and Switching Basics

5.1 What It Is and Why It Matters

Routing is how a packet finds its way from a source to a destination across multiple networks; switching is how frames move between devices on the SAME local network segment. Cloud VPC networking (Part 20) is essentially a managed, software-defined version of these exact concepts — understanding them directly demystifies route tables, security groups vs NACLs, and why some connectivity issues are “routing problems” while others are “firewall problems.”

5.2 Default Gateways and Route Tables

Every host has a routing table telling it, for any destination IP, which interface and next-hop to send the packet to. The **default gateway** is the “send it here if nothing more specific matches” fallback entry — typically your router/firewall, or in a cloud VPC, the subnet’s route table default entry.

```
ip route show                # the host's routing table (Part 01, Section 8.2)
# default via 10.0.1.1 dev eth0      ← default gateway
# 10.0.1.0/24 dev eth0 proto kernel scope link src 10.0.1.15 ← local subnet, no
gateway needed

ip route get 8.8.8.8          # which route/gateway would actually be used to reach
this specific destination
```

In a cloud VPC, a **route table** attached to a subnet defines exactly this: traffic to the VPC’s own CIDR stays local; traffic to `0.0.0.0/0` (everything else) goes to an Internet Gateway (public subnet) or a NAT Gateway (private subnet) — this single route table entry is the actual mechanism that makes a subnet “public” or “private,” not a label.

5.3 ARP: Resolving IP to MAC Addresses

Within a local network segment, devices communicate using MAC addresses (Layer 2), not IP addresses directly. **ARP (Address Resolution Protocol)** is how a host discovers “which MAC address corresponds to this IP address on my local network.”

```
ip neigh show                # the ARP table – cached IP-to-MAC mappings this host
has learned
arp -a                       # older equivalent command
```

WHY THIS MATTERS LESS IN CLOUD ENVIRONMENTS

In traditional on-prem networking, ARP issues (stale cache entries, ARP spoofing) are a real troubleshooting category. In cloud VPCs, the underlying virtual network fabric handles this transparently — you'll rarely debug ARP directly on AWS/Azure/GCP, but understanding it clarifies why a "Layer 2 adjacent" network behaves differently from a routed one, and underpins how container networking (CNI, Part 10) bridges work.

5.4 Security Groups vs Network ACLs (the Conceptual Preview)

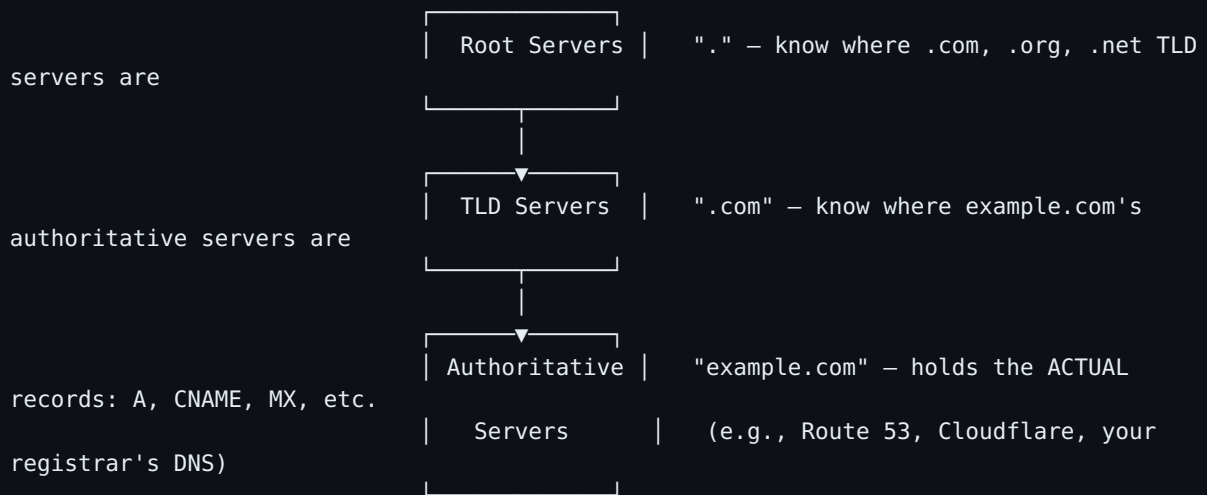
This distinction is fully covered with AWS specifics in Part 20, but the underlying networking concept belongs here: a **stateful** firewall (like an AWS Security Group) automatically allows return traffic for a connection it already approved outbound (or vice versa) without a matching explicit rule; a **stateless** firewall (like an AWS Network ACL) evaluates every packet independently in both directions, requiring explicit rules for both the request AND its response. This is precisely why a stateless NACL misconfiguration is a very common "outbound works, but the response never comes back" bug, while an equivalent Security Group mistake simply blocks the connection outright, more obviously.

6. DNS Deep Dive

6.1 What It Is and Why It Exists

DNS (Domain Name System) translates human-readable names (`api.example.com`) into IP addresses computers actually route traffic to. It is a distributed, hierarchical, heavily-cached system — understanding its architecture explains why DNS changes don't propagate instantly, why "it works on my machine but not in the office" happens, and why DNS is both one of the most common sources of outages AND one of the most powerful tools for traffic control (failover, blue-green cutover, geo-routing).

6.2 DNS Hierarchy and Resolution Process



Full resolution path for a NEW (uncached) query, e.g., a client looking up `api.example.com`:

1. Client asks its configured **recursive resolver** (ISP's resolver, `8.8.8.8`, or a corporate/cloud-internal resolver).
2. Resolver asks a **root server**: "who handles `.com`?" → gets a referral to the `.com` TLD servers.
3. Resolver asks a **TLD server**: "who's authoritative for `example.com`?" → gets a referral to `example.com`'s nameservers (e.g., AWS Route 53's assigned nameservers).
4. Resolver asks the **authoritative nameserver**: "what's the A record for `api.example.com`?" → gets the actual IP address back.
5. Resolver returns the answer to the client AND **caches it** for the record's TTL duration.

```
dig +trace api.example.com          # watch this EXACT process happen live, hop by hop
– the single best command for understanding DNS resolution
```

WHY MOST QUERIES NEVER ACTUALLY TOUCH ROOT/TLD SERVERS

Because of aggressive caching at every layer (OS resolver cache, recursive resolver cache, even some applications' internal caches), the vast majority of real-world DNS queries are answered entirely from cache, never touching the root/TLD/authoritative chain at all. This is also exactly why DNS CHANGES can take time to be visible everywhere — every layer of cache between the authoritative server and the end client has to expire (based on TTL) before it re-queries and sees the new value.

6.3 DNS Record Types

Record	Purpose	Example
A	Hostname → IPv4 address	<code>api.example.com → 203.0.113.10</code>
AAAA	Hostname → IPv6 address	<code>api.example.com → 2001:db8::1</code>
CNAME	Hostname → ANOTHER hostname (alias)	<code>www.example.com → example.com</code>
MX	Mail routing, with priority	<code>example.com → 10 mail.example.com</code>
TXT	Arbitrary text — domain verification, SPF/DKIM email auth	<code>v=spf1 include:_spf.google.com ~all</code>
NS	Delegates a (sub)domain to specific nameservers	<code>example.com → ns-1.awsdns-00.com</code>
SOA	Start of Authority — zone metadata (primary NS, serial, refresh/retry/expire timers)	—
PTR	IP address → hostname (reverse DNS)	<code>10.0.113.203.in-addr.arpa → api.example.com</code>
SRV	Service location: host + port for a named service	Used by things like Kubernetes headless services internally
ALIAS / ANAME (provider-specific)	CNAME-like behavior, but allowed at the zone APEX (root domain) where a real CNAME is disallowed by spec	AWS Route 53 calls this an “Alias record”

COMMON MISTAKE: CNAME AT THE ZONE APEX

The DNS spec disallows a CNAME record at a zone's apex/root (e.g., you cannot make `example.com` itself a CNAME, only subdomains like `www.example.com`) because the apex must also hold other record types (like NS, SOA, MX) which a CNAME would conflict with. This is exactly why cloud providers invented “Alias” (Route 53) or “ANAME”/“flattened CNAME” (other providers) records — a provider-specific way to point a root domain at another hostname (like a load balancer's DNS name) while staying spec-compliant.

6.4 TTL and Caching

TTL (Time To Live), specified in seconds on every DNS record, tells every resolver and cache along the chain how long they're allowed to keep and reuse that answer before re-querying the authoritative server.

```
dig api.example.com          # the TTL is shown directly in the ANSWER section
output
```

TTL value	Tradeoff
Low (e.g., 60s)	Fast propagation of changes, enables quick DNS-based failover — but far more queries hit your authoritative DNS servers (cost, and a brief dependency spike during any outage)
High (e.g., 86400s / 24h)	Much lower query load on authoritative servers, better client-side caching performance — but any change takes up to that long to fully propagate everywhere

PRODUCTION PRACTICE: PLANNING A DNS CUTOVER

Before any planned migration (new load balancer, new provider, new IP), **LOWER** the TTL on the relevant record (e.g., from 3600 to 60) well in advance — often 24-48 hours ahead, to let the OLD high TTL fully expire everywhere first. Only then make the actual change. This ensures that when you do cut over, the new value propagates in under a minute instead of being stuck behind a stale, still-cached 24-hour-old answer on some clients. Forgetting this step is one of the most common causes of “but I already changed the DNS record, why are some users still hitting the old server?” incidents.

6.5 Authoritative DNS in Practice: Route 53 and Friends

Cloud-managed authoritative DNS services (AWS Route 53, Cloudflare DNS, Google Cloud DNS) add features beyond plain static records:

- **Alias records** (Route 53): point a root domain directly at an AWS resource (ALB, CloudFront, S3 website) with automatic IP updates and no extra DNS lookup charge, unlike a CNAME.
- **Health-check-based routing / failover routing**: automatically stop returning a record if its associated health check fails, routing to a backup instead.
- **Weighted routing**: split traffic by percentage across multiple targets (canary releases, A/B infrastructure tests, gradual migrations).
- **Latency-based / geolocation routing**: return different records depending on the querying resolver's location, for geographically distributed infrastructure.

These are all still “just DNS” underneath — fundamentally the same A/CNAME/Alias records, but with the ANSWER computed dynamically per-query instead of being static, which is why DNS-based traffic management is such a common, powerful, and relatively simple tool in a DevOps engineer’s toolkit, well before a full load-balancer-level solution is needed.

6.6 Split-Horizon / Private DNS

Many environments run **split-horizon DNS**: the same hostname resolves differently depending on whether the query originates from inside a private network (e.g., a VPC) or from the public internet — e.g., `db.internal.example.com` might resolve to a private `10.x.x.x` IP only when queried from inside the VPC, and not resolve at all (or resolve to nothing useful) externally. AWS Route 53 Private Hosted Zones and Kubernetes’ internal CoreDNS (Part 10) are both direct implementations of this exact concept, and it’s the standard pattern for keeping internal service names out of public DNS entirely.

7. HTTP and HTTPS Protocol Deep Dive

7.1 What It Is and Why It Exists

HTTP (HyperText Transfer Protocol) is the application-layer protocol underlying virtually all web APIs, web applications, and most microservice-to-microservice communication. Nearly every piece of infrastructure in this series — load balancers, Ingress controllers, API gateways, health checks — operates by inspecting or terminating HTTP. Genuinely understanding its anatomy (not just “GET and POST”) is foundational to debugging almost any production web-facing issue.

7.2 Anatomy of an HTTP Request and Response

HTTP REQUEST:

```
GET /api/users?active=true HTTP/1.1  
Host: api.example.com  
Authorization: Bearer eyJhbGc...  
Accept: application/json  
User-Agent: curl/8.4.0  
  
(request body – empty for GET)
```

← request line: METHOD, PATH+QUERY, VERSION

← headers (key: value pairs)

← blank line separates headers from body

HTTP RESPONSE:

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Content-Length: 142  
Cache-Control: max-age=60  
  
{"users": [{"id": 1, "name": "..."}]}
```

← status line: VERSION, STATUS CODE, REASON PHRASE

← response headers

← response body

```
curl -v https://api.example.com/api/users # -v shows the ACTUAL request and  
response lines/headers exchanged on the wire (Part 01, Section 8.5)  
curl -I https://api.example.com  
# HEAD request – headers only, no body, fast way to check status/headers
```

7.3 HTTP Methods

Method	Semantics	Idempotent?	Has a body?
GET	Retrieve a resource	Yes	No (by convention; some tools allow it, but avoid relying on it)
POST	Create a resource / submit data, non-idempotent action	No	Yes
PUT	Replace a resource entirely	Yes	Yes
PATCH	Partially update a resource	No (by spec, though often implemented idempotently)	Yes
DELETE	Remove a resource	Yes	Usually no
HEAD	Like GET, but headers only, no body	Yes	No
OPTIONS	Discover allowed methods/CORS preflight	Yes	No

WHY “IDEMPOTENT” MATTERS IN PRODUCTION

An idempotent request produces the same end state no matter how many times it's retried (e.g., “set balance to \$100” — repeating it changes nothing further). This is exactly why load balancers, API gateways, and HTTP clients can SAFELY auto-retry a failed **GET**, **PUT**, or **DELETE** on a timeout, but must NOT safely auto-retry a **POST** without additional protection (like an idempotency key) — a retried **POST /charge-card** could charge a customer twice. This single distinction underlies a huge amount of real API design and client retry-logic decisions.

7.4 HTTP Status Codes

Range	Category	Common codes and real meaning
1xx	Informational	101 Switching Protocols (used in the WebSocket upgrade handshake)
2xx	Success	200 OK , 201 Created , 204 No Content (success, deliberately empty body), 206 Partial Content (range requests)
3xx	Redirection	301 Moved Permanently (cache this redirect), 302 Found / 307 Temporary Redirect (don't cache), 304 Not Modified (conditional GET cache hit)
4xx	Client error	400 Bad Request , 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but not allowed), 404 Not Found , 409 Conflict , 429 Too Many Requests (rate limited)
5xx	Server error	500 Internal Server Error , 502 Bad Gateway , 503 Service Unavailable , 504 Gateway Timeout

502 VS 503 VS 504 — A CRITICAL TROUBLESHOOTING DISTINCTION

- **502 Bad Gateway:** the proxy/load balancer successfully connected to the backend, but got an invalid or malformed response (or the connection was abruptly reset) — usually means the BACKEND APPLICATION crashed or returned garbage, not a networking problem.
- **503 Service Unavailable:** the proxy/load balancer couldn't find ANY healthy backend to send the request to at all — all targets are failing health checks, or there's genuinely nothing registered. Points to the backend fleet/target group, not an individual request.
- **504 Gateway Timeout:** the proxy connected to a backend successfully, but the backend took too long to respond and the proxy gave up — points to backend PERFORMANCE (slow query, deadlock, resource exhaustion), not a crash.

These three errors point you toward three completely different investigation paths, and distinguishing them immediately (from the load balancer/Ingress's own logs, not just "the site is down") is one of the fastest ways to cut incident response time.

7.5 Essential Headers

Header	Purpose
<code>Host</code>	Which virtual host/domain this request is for — essential for name-based virtual hosting (many sites, one IP)
<code>Content-Type</code>	The media type of the body (<code>application/json</code> , <code>text/html</code> , <code>multipart/form-data</code>)
<code>Content-Length</code>	Body size in bytes — mismatches here are a classic cause of truncated/hanging requests
<code>Authorization</code>	Credentials: <code>Bearer <token></code> , <code>Basic <base64></code>
<code>Cache-Control</code>	Caching directives: <code>no-cache</code> , <code>max-age=3600</code> , <code>private</code> , <code>public</code>
<code>ETag</code> / <code>If-None-Match</code>	Content fingerprint for conditional requests — enables efficient <code>304 Not Modified</code> responses
<code>X-Forwarded-For</code>	The ORIGINAL client IP, added by each proxy hop — essential when your app sits behind a load balancer and <code>request.remote_addr</code> would otherwise just show the LB's own IP
<code>X-Forwarded-Proto</code>	Tells the backend whether the ORIGINAL request was HTTP or HTTPS — critical when TLS is terminated at the load balancer and the backend itself speaks plain HTTP internally

COMMON PRODUCTION BUG: TRUSTING X-FORWARDED-FOR BLINDLY

`X-Forwarded-For` is just a regular header — any CLIENT can set it to anything they want. Unless your load balancer/proxy is configured to overwrite (not append-trust) this header at the edge, an attacker can spoof their apparent IP trivially, breaking IP-based rate limiting or access control. Always ensure the edge-most proxy strips/overwrites client-supplied values for this header rather than trusting it as-is.

7.6 HTTP/1.1 vs HTTP/2 vs HTTP/3

	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC (built on UDP)
Multiplexing	No — one request per connection at a time (workaround: multiple parallel connections)	Yes — many requests/responses interleaved over ONE connection	Yes, over QUIC streams
Head-of-line blocking	Yes, badly	Reduced (but still exists at the TCP layer)	Eliminated (QUIC streams are independent at the transport level)
Header compression	No	Yes (HPACK)	Yes (QPACK)
Connection setup	TCP handshake + TLS handshake, sequential	Same as 1.1	Combines transport + TLS handshake into fewer round trips

WHY THIS MATTERS FOR INFRASTRUCTURE, NOT JUST BROWSERS

HTTP/2's multiplexing is WHY a modern browser can load dozens of assets over a single connection efficiently, and it's directly relevant when configuring load balancers/Ingress controllers (AWS ALB, NGINX Ingress) — many support HTTP/2 to the client while still speaking HTTP/1.1 to backend targets, and misconfiguration here is a real source of subtle performance or compatibility issues. gRPC (common in microservice-to-microservice communication) is built directly on top of HTTP/2 specifically to get multiplexing and streaming.

7.7 Keep-Alive and Connection Reuse

```
curl -v https://api.example.com/      # look for "Connection: keep-alive" and note
whether curl reuses the connection across multiple requests with -v when hitting the
same host twice
```

Establishing a new TCP + TLS connection for every single HTTP request is expensive (handshake round trips add real latency). **Connection: keep-alive** (the HTTP/1.1 default) lets a client reuse the same underlying TCP connection for multiple sequential requests. This is exactly why connection pooling in HTTP clients (and correctly configured keep-alive timeouts on load balancers/backends) matters enormously for latency and throughput at scale — a mismatched keep-alive timeout between a load balancer and backend is a classic, hard-to-diagnose source of intermittent **502**s (the backend silently closes an “idle” connection the LB still believes is alive, and the next request sent over it fails).

7.8 HTTP Caching

```
Client                                     Server
| — GET /app.js —————> |
| ← 200 OK, Cache-Control: max-age=3600, ETag:"a1" |
| |
| (within 1 hour: serve from LOCAL cache, no request sent at all)
|
| — GET /app.js, If-None-Match: "a1" —————> | (after 1 hour, revalidate)
| ← 304 Not Modified (empty body – still valid!) – | (content unchanged – saves
bandwidth, still a round trip)
```

Directive	Meaning
<code>max-age=N</code>	Cache for N seconds before revalidating
<code>no-cache</code>	MUST revalidate with the server before using a cached copy (misleadingly named — it doesn't mean "don't cache")
<code>no-store</code>	Genuinely never cache this at all (sensitive data)
<code>private</code>	Cacheable only by the end-user's browser, not shared/intermediate caches (CDNs)
<code>public</code>	Cacheable by shared caches (CDNs, proxies) too
<code>immutable</code>	This exact URL's content will never change — skip revalidation entirely until it expires (common for versioned/hashed static asset filenames)

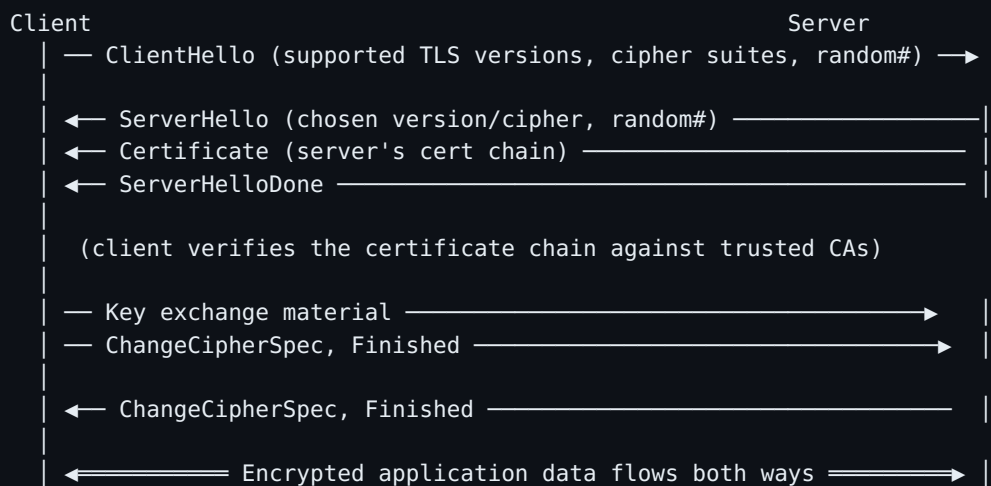
This same caching model is exactly what CDNs (CloudFront, Cloudflare) implement at scale, and it's directly relevant to Part 20's CloudFront coverage and to designing cache headers correctly for any API or static asset layer.

8. SSL/TLS and Certificates

8.1 What It Is and Why It Exists

TLS (Transport Layer Security — the modern successor to SSL; people still say “SSL” colloquially, but SSL itself is deprecated and insecure) provides three guarantees for a connection: **encryption** (nobody in the middle can read the data), **integrity** (nobody in the middle can tamper with it undetected), and **authentication** (you can cryptographically verify you’re actually talking to who you think you are, via certificates). Every HTTPS connection, and an increasing number of internal service-to-service connections (mTLS), depend on this exact mechanism.

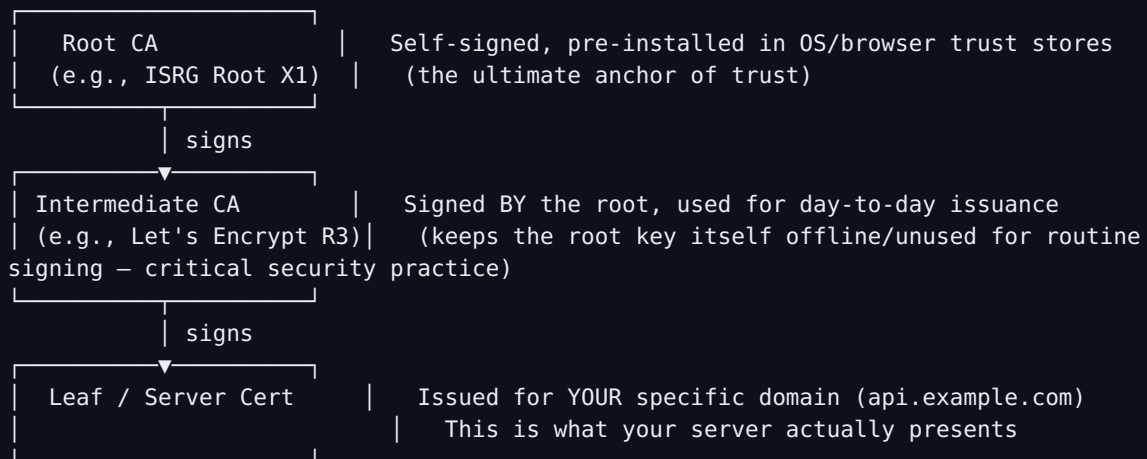
8.2 The TLS Handshake



TLS 1.3 (the current modern standard) streamlines this to effectively **one round trip** before application data can flow, down from two in TLS 1.2 — a meaningful latency improvement at scale, and one reason “which TLS version” is a real, measurable performance and security decision, not just a checkbox.

```
curl -v https://api.example.com 2>&1 | grep -i "SSL\|TLS" # see the negotiated TLS
version and cipher in curl's verbose output
openssl s_client -connect api.example.com:443 -tls1_3 # manually initiate a
handshake and inspect everything exchanged
```

8.3 The Certificate Chain of Trust



When a client connects, the server presents its **leaf certificate** plus the **intermediate certificate(s)** needed to chain up to a root the client already trusts (roots themselves are never sent — they're pre-installed). The client verifies each signature up the chain, confirms the final root is in its trust store, checks the certificate hasn't expired, and checks the hostname matches.

```
# Inspect a live certificate and its full chain
openssl s_client -connect api.example.com:443 -showcerts </dev/null 2>/dev/null | openssl x509 -noout -text

# Quick expiry check – one of the most common production checks you'll run
echo | openssl s_client -connect api.example.com:443 2>/dev/null | openssl x509 -noout -dates
# notBefore=Jan  1 00:00:00 2025 GMT
# notAfter=Apr  1 23:59:59 2025 GMT
```

COMMON MISTAKE: “WORKS IN MY BROWSER, FAILS FROM CURL/A SERVER”

A server that presents ONLY its leaf certificate without the required intermediate certificate(s) often still works in major browsers, because browsers aggressively cache and fetch missing intermediates via AIA (Authority Information Access) chasing — but command-line tools, many programming language HTTP clients, and mobile apps often do NOT do this, and will fail TLS verification outright. Always configure the FULL chain (leaf + intermediates) on the server side, and verify with `openssl s_client -showcerts` specifically, not just a browser padlock icon.

8.4 Certificate Authorities and ACME / Let's Encrypt

A **Certificate Authority (CA)** is a trusted entity (trusted because its root certificate ships pre-installed in browsers/OSes) that verifies domain ownership and issues signed certificates. **Let's Encrypt** made this process free and fully automatable via the **ACME (Automatic Certificate Management Environment)** protocol — the foundation of most modern TLS automation (cert-manager in Kubernetes, Part 11, is built directly on ACME).

```
# certbot – the reference ACME client, for a traditional VM/Nginx setup
sudo certbot --nginx -d api.example.com          # obtains a cert AND auto-configures
Nginx to use it
sudo certbot renew --dry-run                     # test the renewal process without
actually renewing
sudo certbot certificates                        # list currently managed certs
and their expiry

# ACME's actual domain-ownership verification happens via a "challenge":
# HTTP-01 challenge: CA asks you to serve a specific token at http://yourdomain/.well-
known/acme-challenge/<token>
# DNS-01 challenge: CA asks you to create a specific TXT record – the ONLY option for
wildcard certs, and works for infra with no public HTTP endpoint at all
```

WHY DNS-01 MATTERS FOR INTERNAL/WILDCARD CERTS

HTTP-01 challenges require a publicly reachable HTTP endpoint on port 80 — impossible for fully internal services or wildcard certificates (`*.example.com` , which only DNS-01 supports). DNS-01 instead proves ownership by creating a TXT record via your DNS provider's API, which is exactly why cert-manager in Kubernetes commonly integrates with Route 53/Cloudflare APIs directly — it's automating the DNS-01 challenge, not hitting your cluster over the internet at all.

8.5 Self-Signed Certificates

```
# Generate a self-signed cert – useful for local dev/internal-only tooling, NEVER for
anything public-facing
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes \
-subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

A self-signed certificate is signed by itself, not by any CA in anyone's trust store — browsers/clients will show a trust warning unless the certificate is explicitly, manually trusted on that machine. Appropriate for local development and some purely internal mTLS setups with a private, internally-distributed CA (Section 8.7) — never appropriate for anything a real external user connects to.

8.6 Common Certificate Problems

Symptom	Likely cause
<code>NET::ERR_CERT_DATE_INVALID</code> / expired cert errors	Certificate's <code>notAfter</code> date has passed — renewal automation failed or was never set up
<code>NET::ERR_CERT_COMMON_NAME_INVALID</code>	The certificate's CN/SAN doesn't include the hostname being accessed — e.g., cert is for <code>example.com</code> but accessed via <code>www.example.com</code> without that SAN entry
<code>unable to get local issuer certificate</code>	Missing intermediate certificate(s) — Section 8.3's chain problem
<code>certificate signed by unknown authority</code>	Self-signed cert, or an internal/private CA not installed in this client's trust store
Works via browser, fails via <code>curl</code> /API client	Usually the missing-intermediate problem (Section 8.3) — browsers hide this more often than other clients

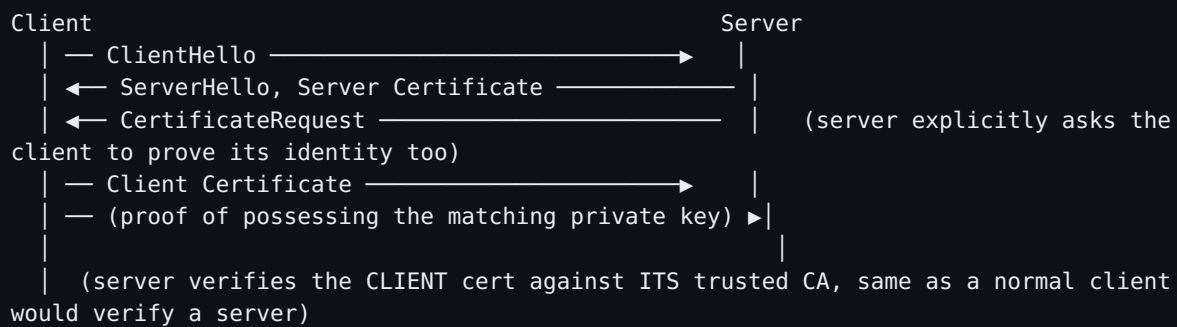
```
# The single most useful certificate-expiry monitoring one-liner – script this as a
scheduled check (Part 01, Section 12)
check_cert_expiry() {
    local host="$1"
    local days_left
    days_left=$(( $(date -d "$(echo | openssl s_client -connect "${host}:443" -
servername "$host" 2>/dev/null | openssl x509 -noout -enddate | cut -d= -f2)" +%s) - $
(date +%s)) / 86400 ))
    echo "$host: ${days_left} days until expiry"
}
check_cert_expiry api.example.com
```

PRODUCTION PRACTICE

Certificate expiry is one of the most common entirely-preventable outages in the industry — set up automated expiry monitoring/alerting (the snippet above, or a dedicated tool/Prometheus blackbox exporter check, Part 22) with a threshold well before expiry (e.g., alert at 30 and again at 7 days remaining), independent of whatever auto-renewal mechanism you have, because auto-renewal CAN silently fail (DNS API credentials rotated, rate limits hit, a configuration change broke the HTTP-01 challenge path) and you want to know before it becomes a customer-facing outage.

8.7 mTLS (Mutual TLS)

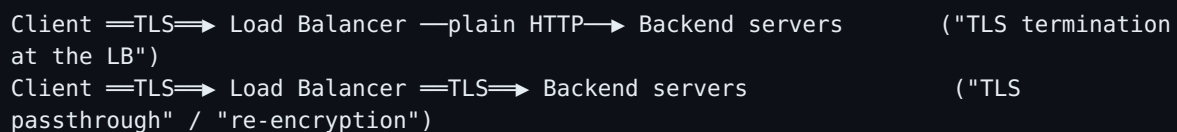
In standard TLS, only the SERVER presents a certificate — the client verifies the server's identity, but the server has no cryptographic proof of who the client is (that's handled separately, e.g., via an API key or bearer token in the request). **mTLS** requires BOTH sides to present and verify certificates, giving strong mutual authentication without relying on a separate application-layer credential.



mTLS is the standard mechanism for securing service-to-service communication inside a service mesh (Istio, Linkerd) and is a common requirement in zero-trust network architectures — every service both proves its own identity and verifies its peer's, rather than trusting anything just because it's "inside the network perimeter." This matters directly for Kubernetes/service mesh topics covered later in this series.

8.8 TLS Termination Points

Where TLS is decrypted matters architecturally:



- **TLS termination at the load balancer:** the LB decrypts, inspects/routes based on plaintext HTTP (enabling L7 features), then sends plain HTTP internally. Simpler certificate management (one place to install/renew certs), but internal traffic is unencrypted — acceptable only within a trusted private network boundary.
- **TLS passthrough / end-to-end TLS:** the LB forwards encrypted bytes without decrypting (pure L4), and the backend itself terminates TLS — necessary when the LB must not (or cannot) see plaintext, or when end-to-end encryption is a compliance requirement.
- **Re-encryption:** the LB terminates the client's TLS connection AND opens a new, separate TLS connection to the backend — gets both L7 capabilities AND encrypted internal traffic, at the cost of extra CPU overhead for the double handshake/encryption.

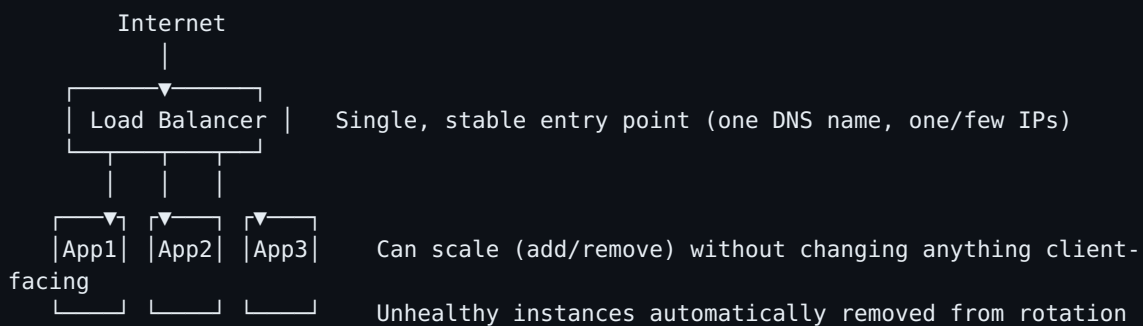
This exact distinction maps directly onto AWS ALB (L7, typically terminates and can re-encrypt to backends) vs NLB (L4, commonly used for passthrough) — covered with full AWS configuration detail in Part 20.

9. Load Balancing and Reverse Proxies

9.1 What They Are and Why They Exist

A **reverse proxy** sits in front of one or more backend servers, accepting client requests on their behalf and forwarding them to an appropriate backend — the client only ever talks to the proxy, never directly to a backend. A **load balancer** is a reverse proxy specifically focused on distributing traffic across multiple backend instances for scalability and availability. In practice, the same piece of software (NGINX, HAProxy, an AWS ALB) is usually doing both jobs simultaneously.

9.2 Why This Architecture Exists



Without this layer, clients would need to know about every individual backend instance directly — impossible to scale, and a single instance failure would be directly visible to clients instead of being silently routed around.

9.3 Load Balancing Algorithms

Algorithm	How it works	Good for
Round robin	Requests distributed sequentially, one to each backend in turn	Simple, works well when backends and requests are roughly uniform
Least connections	New request goes to whichever backend currently has the fewest active connections	Backends with variable request duration (some requests take much longer than others)
IP hash	Client IP determines which backend it's always routed to	Session affinity without needing shared session storage (though generally weaker than real sticky sessions — Section 9.4)
Weighted (any of the above)	Backends get a proportional share based on an assigned weight	Mixed-capacity backend instances (e.g., during a gradual instance-type migration), or canary traffic splitting
Least response time	Combines connection count AND actual observed latency	Latency-sensitive services with heterogeneous backend performance

9.4 Health Checks

A load balancer must know which backends are actually healthy before routing to them — this is the single most important piece of load balancer configuration to get right.

```
Active health check: LB —periodic GET /health→ Backend
                    LB ← 200 OK (healthy) — Backend
                    (LB removes backend from rotation after N consecutive failures;
                     re-adds after N consecutive successes — both thresholds matter)
```

Setting	What it controls
Health check path	Which endpoint is probed — should check REAL dependencies (DB connectivity, critical downstream services), not just “is the process alive”
Interval	How often the check runs
Timeout	How long to wait for a response before considering that check a failure
Healthy threshold	Consecutive successes needed before marking a backend healthy (back in rotation)
Unhealthy threshold	Consecutive failures needed before marking a backend unhealthy (removed from rotation)

COMMON ANTI-PATTERN: A HEALTH CHECK THAT ALWAYS RETURNS 200

A `/health` endpoint that just returns `200 OK` unconditionally (not actually checking database connectivity, disk space, or critical dependencies) defeats the entire purpose — the load balancer will happily keep sending traffic to an instance that's fully broken internally, because the health check itself was never actually testing anything meaningful. A genuinely useful health check verifies the specific things that would make this instance unable to correctly serve requests, while staying FAST (health checks run frequently; an expensive check becomes its own load problem) and avoiding cascading failures (a health check shouldn't itself fail the whole fleet if ONE shared downstream dependency has a blip — consider separate “liveness” vs “readiness” semantics, covered in full for Kubernetes in Part 13).

9.5 Session Affinity (“Sticky Sessions”)

Some applications store session state in-memory on a specific backend instance, requiring all of a given user's requests to consistently reach the SAME backend. Load balancers support this via a cookie the LB itself injects (`AWSALBAPP` , or NGINX's `ip_hash` /cookie-based affinity) to pin a client to one backend for the session's duration.

WHY THIS IS USUALLY AN ANTI-PATTERN TO DESIGN AROUND

Sticky sessions fundamentally fight against the elasticity and failure-tolerance load balancing is supposed to provide — if that specific backend instance dies, every user pinned to it loses their session state entirely, and uneven user activity can create unbalanced load across instances despite the LB's best effort. The modern, more scalable pattern is to keep backends fully STATELESS and externalize session state to a shared store (Redis/ElastiCache, Part 20) that any backend instance can read — eliminating the need for session affinity entirely, and letting any request go to any healthy instance.

9.6 API Gateways

An API gateway is a specialized reverse proxy sitting at the edge of a system of APIs/microservices, typically adding capabilities beyond basic load balancing: authentication/authorization enforcement, rate limiting, request/response transformation, API versioning, and aggregating multiple backend calls into one client-facing response. AWS API Gateway, Kong, and an Nginx/Envoy configuration with custom logic are all implementations of this same pattern — the architectural role is the same even when the specific product differs, and this concept connects directly to the Ingress/Gateway API coverage in Part 10.

10. Network Troubleshooting Methodology

10.1 The Layered Diagnostic Approach

Network issues should be diagnosed layer by layer, from the bottom up (or systematically working through the OSI layers from Section 2), because a failure at a lower layer makes everything above it fail too, in ways that can LOOK like a higher-layer problem if you don't check in order.

```
Layer 7 (App/HTTP)    "Getting a 500 error" / "wrong response body"    — curl -v, check response code/body
Layer 4 (TCP/UDP)     "Connection refused" / "connection reset"      — nc -zv, ss -tan
Layer 3 (IP/Routing)  "No route to host" / packets not arriving at all — ping, traceroute/mtr, ip route
DNS (sits alongside) "Could not resolve host"                      — dig, nslookup
```

10.2 The Systematic Sequence

```
# 1. Does the name even resolve, and to what?
dig +short api.example.com

# 2. Is there a network path to that IP at all? (L3)
ping -c 4 <resolved-ip>          # note: ICMP can be blocked even when the
service itself is fine — inconclusive alone
mtr <resolved-ip>                # better: shows exactly where along the path things
degrade/stop

# 3. Is the specific port open? (L4)
nc -zv <resolved-ip> 443

# 4. Does the TLS handshake succeed? (if applicable)
openssl s_client -connect <resolved-ip>:443 -servername api.example.com </dev/null

# 5. Does the actual HTTP request succeed, and with what response? (L7)
curl -v https://api.example.com/health

# 6. If it's intermittent/under load, capture packets to see the EXACT failure pattern
sudo tcpdump -i any host <resolved-ip> and port 443 -w capture.pcap
```

WHY THIS ORDER MATTERS

Jumping straight to `curl -v` when DNS itself is broken wastes time interpreting a confusing error that's really just "couldn't resolve the name" dressed up as an HTTP-layer problem. Working the layers in order means each step either confirms that layer is fine

(move up) or identifies exactly where to focus (stop and dig deeper there) — this is the single most useful discipline in network troubleshooting, and it's exactly the model referenced again in Part 10 for Kubernetes-specific networking issues.

10.3 Common Production Scenarios

Scenario: Intermittent connection failures under load

```
ss -s                                # check total socket counts, especially TIME_WAIT
ss -tan state time-wait | wc -l      # is ephemeral port exhaustion the actual cause?
cat /proc/sys/net/ipv4/ip_local_port_range # the available ephemeral port range on
this host
```

Diagnosis: High connection churn (many short-lived outbound connections) can exhaust the ephemeral port range or pile up `TIME_WAIT` sockets faster than the OS can recycle them, causing new outbound connections to intermittently fail. Fixes include connection pooling/reuse in the client, tuning `net.ipv4.tcp_tw_reuse`, or widening the ephemeral port range — this is also a very common issue inside NAT Gateways at high connection-rate scale (AWS NAT Gateway port exhaustion, covered with AWS specifics in Part 20).

Scenario: Works from one machine, fails from another

```
# Compare DNS resolution between the two machines FIRST – this differs more often than
people assume
dig api.example.com                # run on both machines, compare results

# Compare the actual network path
mtr api.example.com                # run on both, look for where they diverge

# Compare TLS/cert trust specifically if it's a TLS error
openssl s_client -connect api.example.com:443 </dev/null 2>&1 | grep "Verify return
code"
```

Diagnosis: Usually one of: different DNS resolvers returning different answers (split-horizon DNS, Section 6.6, or just different caches with different TTLs remaining), a firewall/security-group rule that only applies to one machine's source IP/subnet, or a missing CA certificate in one machine's trust store (Section 8.6).

Scenario: Load balancer shows healthy targets, but users still report errors

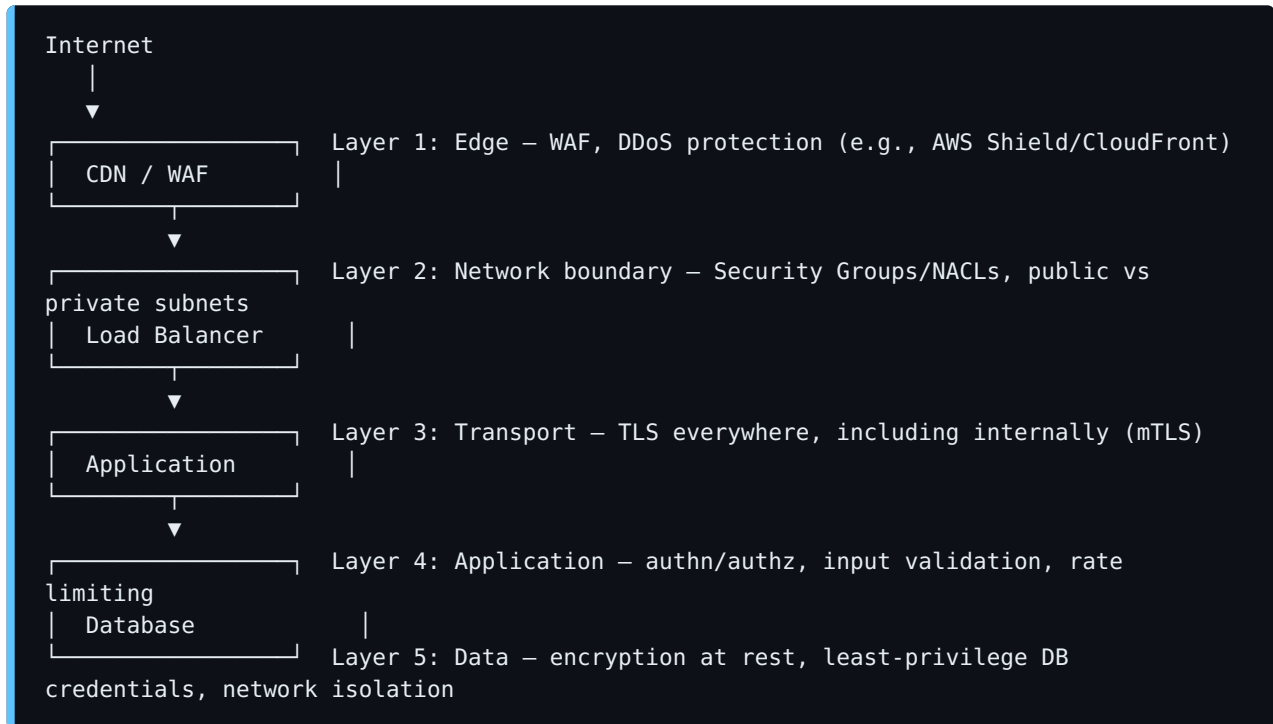
```
# The health check path and the ACTUAL user-facing path may behave completely
differently
curl -v https://api.example.com/health    # what the LB checks
curl -v https://api.example.com/api/users # what users actually hit
```

Diagnosis: This is almost always the “health check doesn’t check anything meaningful” anti-pattern from Section 9.4 — the shallow health check endpoint succeeds while a REAL dependency (database, downstream API, disk space) that only the actual user-facing endpoints touch is failing. Fix the health check to actually probe the dependencies that matter, so the load balancer’s view of “healthy” matches reality.

11. Network Security Fundamentals

11.1 Defense in Depth

Production network security relies on multiple independent layers, so that a failure in any single one doesn't mean total compromise:



11.2 Common Network-Level Attacks and Mitigations

Attack	What it is	Primary mitigation
DDoS (Distributed Denial of Service)	Overwhelming a target with traffic from many sources to exhaust capacity	CDN/edge absorption (CloudFront + Shield, Cloudflare), auto-scaling, rate limiting
Man-in-the-middle (MITM)	Intercepting/altering traffic between two parties	TLS everywhere (Section 8), certificate pinning for high-value clients, HSTS
DNS spoofing/cache poisoning	Tricking a resolver into caching a fraudulent answer	DNSSEC, using trusted/validating resolvers
Port scanning / reconnaissance	Probing for open ports and running services	Minimal exposed surface (only necessary ports open), fail2ban/rate limiting on exposed services (Part 01, Section 15.3)
ARP spoofing	Forging ARP replies to intercept local-network traffic	Mostly mitigated by cloud VPC network fabric design; relevant on-prem
SYN flood	Sending many SYN packets without completing the handshake, exhausting connection-tracking resources	SYN cookies (kernel-level mitigation, on by default on modern Linux), upstream DDoS protection

11.3 TLS Everywhere and HSTS

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

HSTS (HTTP Strict Transport Security) is a response header telling browsers “never attempt a plain HTTP connection to this domain again, even if the user types `http://` or clicks an old link — automatically upgrade to HTTPS, and refuse to proceed past certificate errors.” This closes the window for an attacker to intercept an initial unencrypted request and strip/downgrade the connection before HTTPS is established (a real historical attack class called SSL-stripping).

11.4 VPNs and Private Connectivity, Conceptually

A VPN (Virtual Private Network) creates an encrypted tunnel between two points over an untrusted network (typically the public internet), making remote resources appear as if they’re on the same private network. In cloud infrastructure, this concept appears as **Site-to-Site VPN** (connecting an on-prem data center to a cloud VPC) and **Client VPN** (individual engineers connecting to private cloud resources) — both covered with AWS-

specific configuration in Part 20. The architectural point: a VPN doesn't grant NEW permissions, it extends network REACHABILITY — you still need proper authentication/authorization on top.

11.5 Zero Trust, Conceptually

Traditional “castle and moat” network security assumes anything inside the perimeter (the VPC, the office network) is implicitly trusted. **Zero trust** architecture assumes NOTHING is trusted by network location alone — every request, whether from “inside” or “outside,” must be authenticated and authorized on its own merits, typically via mTLS (Section 8.7) and strong service identity, not just “it came from inside the VPC so it must be fine.” This is the underlying philosophy behind service meshes and is increasingly the default assumption in modern cloud-native security design, directly relevant to the Kubernetes NetworkPolicy and RBAC coverage later in this series.

12. Hands-On Labs

LAB ENVIRONMENT

Labs need only a Linux machine with standard networking tools (`curl` , `dig` , `openssl` , `tcpdump`) — a free-tier cloud VM or local VM is fine. Lab 5 benefits from access to a domain you control (even a free/test domain) for real DNS changes.

Lab 1 — Subnetting by Hand, Then Verify

1. Given `172.16.0.0/20` , calculate by hand: total addresses, usable hosts, the network address, and the broadcast address.
2. Split it into 8 equal subnets — calculate the new prefix length and list all 8 resulting CIDR blocks.
3. Verify every answer using `ipcalc` or `sipcalc` , and reconcile any discrepancies.

Lab 2 — Watch DNS Resolution Happen

1. Run `dig +trace` against a real public domain and identify each hop: root servers, TLD servers, authoritative servers.
2. Run `dig` against several different resolvers (`8.8.8.8` , `1.1.1.1` , your ISP's default) for the same domain and compare TTLs and answers.
3. Pick a domain, note its current TTL, and (if you control a test domain) practice the safe cutover pattern from Section 6.4: lower the TTL, wait, then change the record, and observe propagation timing with repeated `dig` queries.

Lab 3 — TLS Handshake and Certificate Chain Inspection

1. Use `openssl s_client -showcerts` against three different public HTTPS sites and compare their certificate chains (issuer, validity period, SAN entries).
2. Deliberately connect to a site using an outdated/weak protocol version flag (e.g., `-tls1` if still supported anywhere, or simulate by testing against a known-strict site) and observe the handshake failure — explain exactly which part of the handshake failed and why.
3. Generate a self-signed certificate locally (Section 8.5), start a simple HTTPS server with it (e.g., `python3 -m http.server` doesn't support TLS natively — use `openssl s_server` or a quick NGINX config), and observe the exact trust error a client gives you when connecting — then explicitly trust it locally and reconnect successfully.

Lab 4 — HTTP Status Code and Header Investigation

1. Using `curl -v`, generate and observe examples of at least five different status codes from real or local test endpoints (200, 301/302, 404, 429 if you have access to a rate-limited API, 500).
2. Set up a locally-run reverse proxy (NGINX or a small script) in front of a backend you control; deliberately misconfigure it to produce a `502` (point it at a closed port) and a `504` (point it at a backend that sleeps longer than the proxy's timeout) — confirm you can distinguish the two from the proxy's own error log.
3. Inspect `Cache-Control` / `ETag` behavior on a real static asset (e.g., from any CDN-backed site) using repeated `curl -v` requests and observe a `304 Not Modified` response when you pass the correct `If-None-Match` header.

Lab 5 — Full-Stack Network Troubleshooting Simulation

1. Stand up a simple backend service on a cloud VM, with a host firewall initially allowing all relevant traffic.
 2. Have a partner (or do this to yourself, “blind,” by writing a script that randomly introduces ONE issue) break ONE of: DNS resolution (point a hostname at the wrong IP), the firewall (block the service port), the TLS cert (let it expire or use a mismatched hostname), or the application itself (stop the service).
 3. Using ONLY the systematic sequence from Section 10.2 — not guessing — diagnose which layer is broken and produce a short written incident summary: symptom, the exact diagnostic command that isolated the cause, root cause, and fix.
-

13. Command & Reference Cheat Sheet

13.1 DNS

Command	Purpose
<code>dig +short <host></code>	Quick resolution
<code>dig +trace <host></code>	Full resolution path, root to authoritative
<code>dig @<resolver> <host></code>	Query a specific resolver directly
<code>dig <host> MX / TXT / NS</code>	Query a specific record type

13.2 Connectivity

Command	Purpose
<code>ping -c 4 <host></code>	Basic L3 reachability (ICMP)
<code>mtr <host></code>	Continuous traceroute + ping, best general-purpose path tool
<code>nc -zv <host> <port></code>	TCP port reachability check
<code>ss -tan state <state></code>	Live connections filtered by TCP state

13.3 HTTP/TLS

Command	Purpose
<code>curl -v <url></code>	Full request/response trace
<code>curl -I <url></code>	Headers only (HEAD request)
<code>curl -o /dev/null -s -w "..."</code>	Timing breakdown (DNS/connect/TLS/TTFB/total)
<code>openssl s_client -connect host:443 -servername host</code>	Manual TLS handshake inspection
<code>openssl x509 -noout -dates -enddate</code>	Certificate validity dates

13.4 Subnetting Quick Reference

CIDR	Hosts	CIDR	Hosts
/24	254	/28	14
/25	126	/29	6
/26	62	/30	2
/27	30	/32	1 (host route)

14. Production Readiness Checklist

- ☐ Certificate expiry monitoring/alerting is independent of auto-renewal automation (Section 8.6) — alerts fire even if renewal silently fails
- ☐ Full certificate chain (leaf + intermediates) is configured, verified with `openssl s_client -showcerts`, not just a browser check
- ☐ HSTS is enabled on all public HTTPS endpoints
- ☐ DNS TTLs are set deliberately per record — low enough for planned failover/migration flexibility, high enough to avoid unnecessary query load
- ☐ Load balancer health checks probe REAL dependencies, not just “process is alive”
- ☐ Health check healthy/unhealthy thresholds and intervals are tuned to the service’s actual failure/recovery characteristics
- ☐ `X-Forwarded-For` / `X-Forwarded-Proto` are set and trusted correctly end to end (overwritten at the true edge, not blindly trusted from clients)
- ☐ Backends are stateless where possible; session state lives in a shared store, not sticky-session-dependent in-memory state
- ☐ Security groups/firewalls follow least-privilege (only required ports/sources allowed) — reviewed periodically for drift
- ☐ Internal service-to-service traffic uses TLS (or mTLS where strong identity matters), not plaintext, even “inside” the private network
- ☐ A documented, systematic network troubleshooting sequence (Section 10.2) is known by the team, not improvised per-incident
- ☐ DDoS/edge protection (CDN, WAF, rate limiting) is in place for any public-facing endpoint